

NOEC

Cara Lewis

August 2026

1 Gameplay Mechanics

1.1 Reading from the ADCs

The ADCs have a resolution of 1024 bits, are then mapped linearly within the given range for the parameter.

1.2 Web controls

The page at /web-controls handles user interaction with ajax post requests to /user-input, using the results of this call to update the UI in much the same way as the main page does. The ui is also created with an initial ajax post request to retrieve ui-config.yaml

1.3 Event Graphs

Each graph displays the probability as calculated for those respective parameter values, with a bin width of 0.2 GeV, multiplied by a normal distribution (centered at 2.3 with a width of 1) to represent the beam, and an event rate multiplier. D3's "step-after" interpolation is used to achieve bins, with an adjustment of half a bin width so that the centre of the bin is at the point it describes.

1.3.1 "True" event line

The blue event line represents the measured values that the user is modelling. If noise mode is activated, each displayed point in this distribution may be increased or decreased from its true value by up to 10%. If slowed data mode is active, the true event line will start at zero, with random increases from within the distribution of up to 0.2, 0.02 and 0.1 for muon electron and anti-electron neutrinos respectively, multiplied by the event rate.

1.3.2 ML event line

The red event line shows the current event distribution found by the gradient descent algorithm using the displayed true values.

1.4 Likelihood Calculation

Likelihood as calculated using the Pearson log likelihood:

$$l = \sum [(p - t)^2 / t]$$

Where p is the value predicted by the user, and t is the displayed true value, including any noise. This is then converted into a user friendly number, L , using

$$L = 100/e^{l/2}$$

which is then rounded to the nearest integer for display.

1.5 2-D likelihood scans

Each scan displays the last 1000 found values (or all previous found values if less than 1000 ticks have passed), where the likelihood found at each point is represented by a colour in the interpolated range from red (`#FF0000`) to lime (`#00FF00`). This is achieved using an array of svg circle objects which are updated at each tick with new positions and values.

1.6 Scoring system

Scores are calculated using

$$s = \min((300 - t), 1)L_T n m o$$

Where t is the time taken in seconds, L_T is the display likelihood of the model when compared to the true, noiseless values, and n , m and o are the noise and machine learning and slowed data multipliers, which are 1.5, 2 and 2 respectively if those modes are active, and 1 otherwise.

Scores can (but do not have to be) be saved alongside a name and the settings active when the score was obtained, and will be displayed on the leaderboard at `/leaderboard`.

1.7 Machine Learning

The machine learning section of the page is only displayed when the ML switch is on. It displays an event plot for the current parameter values provided by the ML algorithm, the current likelihood, whether the algorithm is in progress or complete, a trace plot for the value of likelihood, and three two-dimensional parameter maps used to show how the algorithm moves around the space. The specifics of how parameters are calculated, how stopping thresholds are reached, and what is displayed on the two-dimensional parameter maps depends on the algorithm that is selected using the ML Mode switch.

1.7.1 Gradient Descent

When ML mode is set to GD, the machine learning plot displays the intermediate steps of a gradient descent algorithm, slowed to 10 steps per second. The initial values given for each parameter are selected randomly from within the ADC range, and then mapped to the parameters allowed range by the cost function. The cost function calculates the Pearson log likelihood for the event distribution produced for the given parameters for the `numu`, `nue`, and `nue-b` distributions. The gradient for the cost function is then determined by taking a central limit approximation for each parameter, treating the others as constant. The event oscillation graph and likelihood are calculated and displayed for each step, and the two-dimensional parameter maps display an arrow from the previous position in parameter space to the current one, coloured from red (0%) to green (100%) to show the current likelihood.

The algorithm stops once the displayed likelihood is 100%, or once the displayed likelihood has been over 95 for 500 steps. If the algorithm does not stop within 600 steps, new random values are set for each of the parameters, and the gradient descent starts again.

1.7.2 Markov Chain Monte Carlo

When ML mode is set to MCMC, ten independent walkers perform a random-walk metropolis MCMC algorithm, each taking 10 steps per second. Each walker is given a random starting point with the parameter space, and selects a new step randomly by sampling from a gaussian distribution centred

on its current position, adding it to the chain based on an acceptance threshold calculated using the negative log likelihood multiplied by a bounded uniform prior. The oscillation event graph and likelihood are then calculated using the set of parameters with the highest likelihood at that step. The machine learning plot displays the last 100 (or all if there are less than 100) steps in each walker's markov chain, colour-coded from red for the newest steps, to blue for the oldest.

The algorithm stops when there is at least one walker with parameters that give a likelihood of greater than 98% and the average likelihood over all the walkers is greater than 80%. There is no need to ever restart the algorithm at a new starting point, as eventually the markov chain will converge.

2 Aims and Configurations

The following describes some of the ways that the game could be configured to emphasise different aspects of parameter fitting or neutrino oscillation, and a suggested audience for each configuration. These are not intended to be prescriptive.

2.1 Machine Learning Match

Demonstrate Gradient Descent and Markov Chain Monte Carlo fitting, developing a rough intuition for these two methods and their respective uses. To allow the player to experience both algorithms, the game should be played twice, once in GD mode and once in MCMC mode.

Intended Audience: Ages 13+

2.1.1 Settings

Parameters

Dm32	ON
Th23	ON
dcp	ON

Modes

Noise	OFF
Slow Load	OFF
ML	ON
ML Mode	GD, MCMC

2.1.2 Discussion Points

- Did you beat the algorithm? Do you think you could beat the algorithm on real data? Why?
- What do you think these algorithms could be used on outside particle physics?
- What makes these two algorithms different? Which do you think is the most useful?
- Which algorithm do you think was quicker? Why? Do you think this would always be the case?
- The MCMC algorithm gives us a lot of extra information - what do you think this could be used for?
- sometimes the Gradient descent algorithm gets stuck in an area with a low likelihood - why do you think this happens? Is there anything we could do to fix it?
- When we analyse actual detector data, there are a lot more than 3 parameters - How many do you think there are? Do you think this will make it harder for the algorithms?

- If computer algorithms can find the answers, do we still need physicists? Why/ Why not?
- What do you think the day in the life of a particle physicist looks like? Do you think you would like to be a particle physicist?

2.1.3 Extensions & Adaptions

To reduce time taken, and allow more players to get a turn, a different player could play each stage of this set up. An even shorter adapted version could just demonstrate one algorithm, and discuss why these algorithms are used in HEP.

To extend the game, add in noise or slow data, to demonstrate how these real world factors change the effectiveness of these algorithms.

2.2 Detector Difficulties

Explain some of the difficulties physicists face with detector data. If possible, the game should be played 3 times - one without any additional modes, one with noise, and one with both noise and slow loading data to simulate real-time events.

Intended Audience: Ages 10+

2.2.1 Settings

Parameters

Dm32	ON
Th23	ON
dcp	ON

Modes

Noise	OFF, ON, ON
Slow Load	OFF, OFF, ON
ML	OFF

2.2.2 Discussion Points

- Did the noise make it more difficult? Do you think noise could cause physicists to find a wrong value for these parameters? How do you think physicists deal with this?
- What do you think might cause noise in the detectors? What could we do to reduce this?
- How long did it take to get enough data to find your fit? How long do you think it will take to be able to find these parameters once DUNE starts running?

2.2.3 Extensions & Adaptions

To reduce time taken, and allow more players to get a turn, a different player could play each stage of this set up. Alternatively, each mode could be turned on sequentially at the start, so that the player can get an idea of the difficulty these factors add, while only playing the game once.

To extend this activity, add in a machine learning algorithm to show how they deal with these real world factors.

2.3 Visualising Charge-Parity violation

Introduce parameter fitting simply using only one parameter, and discuss the effect of δ_{CP} and why we want to measure its value.

Intended Audience: Ages 7+, anyone with an interest in matter/anti-matter asymmetry

2.3.1 Settings

Parameters

Dm32	OFF
Th23	OFF
dcp	ON

Modes

Noise	OFF
Slow Load	OFF
ML	OFF

2.3.2 Discussion Points

Parameter Fitting Discussion

- How would you know if you are close to the right answer?
- Why do you think we want to know about this parameter?
- When we analyse actual detector data, there are lots of dials instead of just 1 - How many do you think there are? Do you think it would be easy to find the right answer with that many dials?
- How could we make it easier to find the right answer without having to do it all ourselves?

Charge Parity Violation Discussion

- What do different sizes of δ_{CP} make the graphs look like (big difference between neutrinos and antineutrinos in the middle)
- What kind of particle is missing from our detector information? Why are muon anti-neutrinos not displayed?

2.3.3 Extensions & Adaptions

To extend this activity, add an additional parameter or parameters, and discuss how this changes the difficulty. Alternatively, keep the same number of parameters and add a machine learning algorithm , and discuss why they are used in HEP.